

epochpost

epochpost 是一个基于 sdf_xarray 的 EPOCH PIC 通用后处理脚本。核心思路是：代码只维护一份，日常只修改 epochpost_input.json 来描述要画什么图。

仓库建议保持以下结构：

```
epochpost/  
    epochpost.py  
    epochpost_input.json  
    README.md
```

最常用的运行方式：

```
python epochpost.py epochpost_input.json
```

如果配置文件就在当前目录且名称为 epochpost_input.json，也可以直接：

```
python epochpost.py
```

本程序用于 EPOCH/PIC 的 sdf 文件后处理，底层使用 sdf_xarray。当前版本已经把程序逻辑和每个算例的输入参数分开：

- epochpost.py: 通用后处理模块，原则上不再为每个 run 修改。
- epochpost_input.json: 每个算例自己的后处理输入，只描述“数据在哪里、要画什么图”。

建议以后每个 PIC 算例目录只保存一份自己的 epochpost_input.json。

1. 推荐目录结构

例如：

```
run7/  
    input.deck  
    epochpost_input.json  
    Data/  
        sourcexyspecies0001.sdf  
        sourcexyspecies0002.sdf  
        ...  
        foilparticles0001.sdf  
        ...  
    Data/postprocess/          # 程序自动创建
```

而通用 Python 程序可以放在统一位置，例如：

```
~/python_lib/epochpost.py
```

这样不同 run 之间只需要复制和修改 epochpost_input.json。

2. 运行方法

方法 A: 命令行直接运行

在算例目录中:

```
python ~/python_lib/epochpost.py epochpost_input.json
```

如果当前目录存在 `epochpost_input.json`, 也可以直接:

```
python ~/python_lib/epochpost.py
```

程序会依次尝试:

1. `epochpost_input.json`
2. `post_input.json` (兼容旧版本)
3. `input.json` (兼容旧版本)

也可以传入任意其他 JSON 文件名:

```
python ~/python_lib/epochpost.py run7_post.json
```

方法 B: 作为 Python 模块调用

```
from epochpost import run_config

run_config("epochpost_input.json")
```

这也是以后打包成 Python library 后推荐保留的接口。

3. 最简 JSON

如果你的数据就在 `Data/`, 激光波长为 800 nm, 那么 `settings` 可以完全省略, 最简输入可以只有:

```
{
  "tasks": [
    {
      "type": "spatial",
      "name": "density_10fs",
      "prefix": "sourcexyspecies",
      "variable": {
        "contains": ["Number_Density", "Electron_foil"]
      },
      "axes": ["x", "y"],
      "cuts": {"z": 0.0},
      "select": {"mode": "time", "time_fs": 10.0},
      "animate": false,
      "normalization": {
        "kind": "density_nc",
        "label": "$n_e/n_c$"
      }
    }
  ]
}
```

```

    },
    "scale": "log"
  }
]
}

```

任务中的 `enabled` 如果省略，默认视为 `true`。

4. 全局 settings

完整形式为：

```

"settings": {
  "data_dir": "Data",
  "out_dir": null,
  "laser_wavelength_nm": 800.0,
  "dpi": 180,
  "space_unit": "nm",
  "space_factor": 1000000000.0,
  "time_unit": "fs",
  "time_factor": 1000000000000000.0
}

```

下一次更新时，可以把图片的格式，尺寸等进行更新

含义：

参数	含义	默认值
<code>data_dir</code>	SDF 文件目录	Data
<code>out_dir</code>	输出目录；null 时为 <code><data_dir>/postprocess</code>	null
<code>laser_wavelength_nm</code>	激光波长，用于计算临界密度 <code>nc</code>	800
<code>dpi</code>	PNG 输出分辨率	180
<code>space_unit</code>	图上的空间单位标签	nm
<code>space_factor</code>	SDF 中 <code>m</code> 转换到绘图单位的乘数	1e9
<code>time_unit</code>	时间单位标签	fs
<code>time_factor</code>	SDF 中 <code>s</code> 转换到绘图单位的乘数	1e15

重要：相对路径以 `epochpost_input.json` 所在目录为基准，而不是以 Python 库文件所在目录为基准。

5. 文件选择：单时刻、多个时刻、单编号、多个编号

5.1 最新文件

```

"select": {
  "mode": "latest"
}

```

```
}
```

5.2 单个 SDF 编号

```
"select": {  
  "mode": "number",  
  "number": 78  
}
```

5.3 多个 SDF 编号: 直接 list

```
"select": {  
  "mode": "number",  
  "number": [20, 40, 60, 78]  
}
```

5.4 多个 SDF 编号: arange

JSON 不能直接写 `np.arange()`, 因此程序提供等价写法:

```
"select": {  
  "mode": "number",  
  "number": {  
    "arange": [10, 79, 5]  
  }  
}
```

等价于 Python:

```
np.arange(10, 79, 5)
```

即选择:

10, 15, 20, ..., 75

`arange` 的 `stop` 不包含在结果中, 与 NumPy 完全一致。

5.5 单个物理时刻

```
"select": {  
  "mode": "time",  
  "time_fs": 10.0  
}
```

程序会自动寻找与 10 fs 最近的 SDF 文件。

5.6 多个物理时刻: list

```
"select": {  
  "mode": "time",
```

```
    "time_fs": [2.0, 4.0, 6.0, 8.0, 10.0]
}
```

5.7 多个物理时刻: arange

```
"select": {
    "mode": "time",
    "time_fs": {
        "arange": [2.0, 10.1, 2.0]
    }
}
```

等价于:

```
np.arange(2.0, 10.1, 2.0)
```

5.8 linspace

```
"time_fs": {
    "linspace": [2.0, 10.0, 5]
}
```

等价于:

```
np.linspace(2.0, 10.0, 5)
```

结果为:

2, 4, 6, 8, 10 fs

linspace 包含终点。

5.9 Python range 风格

```
"number": {
    "range": [10, 79, 5]
}
```

等价于:

```
range(10, 79, 5)
```

只适合整数编号。

6. spatial: 二维空间分布

用于某一时刻或多个时刻的二维场、密度、电流等空间分布。

例如:

```

{
  "type": "spatial",
  "name": "foil_density_xy",
  "prefix": "sourcexyspecies",
  "variable": {
    "contains": ["Number_Density", "Electron_foil"]
  },
  "axes": ["x", "y"],
  "cuts": {
    "z": 0.0
  },
  "select": {
    "mode": "time",
    "time_fs": [2, 4, 6, 8, 10]
  },
  "animate": false,
  "xlim": [320, 760],
  "ylim": [-750, 750],
  "normalization": {
    "kind": "density_nc",
    "label": "$n_e/n_c$"
  },
  "scale": "log",
  "vmin": 0.01,
  "vmax": 200,
  "cmap": "magma",
  "save_data": true
}

```

多个 time_fs 或多个 number 时，程序会：

1. 每个时刻/编号分别保存一张 PNG；
2. 再生成一张统一 colorbar 的多面板比较图；
3. save_data=true 时把所有结果保存到一个压缩 .npz 中。

二维动画

```

{
  "type": "spatial",
  "name": "density_animation",
  "prefix": "sourcexyspecies",
  "variable": {"contains": ["Number_Density", "Electron_foil"]},
  "axes": ["x", "y"],
  "cuts": {"z": 0.0},
  "animate": true,
  "time_range_fs": [0.0, null],
  "stride": 5,

```

```

    "max_frames": 100,
    "fps": 12,
    "normalization": {"kind": "density_nc"},
    "scale": "log"
}

```

其中:

- `time_range_fs`: [0, null]: 从 0 fs 到最后一个文件;
- `stride`: 5: 每隔 5 个文件取一帧;
- `max_frames`: 最多保留多少帧;
- `fps`: GIF 帧率。

7. xt: 空间-时间图

适合从不同 SDF 文件中提取同一条空间线, 例如:

- $J_y(x, t)$
- $J_x(x, t)$
- 密度 $n_e(x, t)$
- 某个固定 y, z 截面上的场随时间演化

示例:

```

{
  "type": "xt",
  "name": "foil_Jy_xt_y0",
  "prefix": "sourcexspecies",
  "variable": {
    "contains": ["Jy", "Electron_foil"]
  },
  "line_axis": "x",
  "cuts": {
    "y": 0.0,
    "z": 0.0
  },
  "time_range_fs": [0.0, null],
  "stride": 1,
  "space_range": [320, 760],
  "normalization": {
    "kind": "current_j0",
    "label": "$J_y/(e n_c c)$"
  },
  "scale": "symlog",
  "linthresh": 0.02,
  "cmap": "RdBu_r",
}

```

```

    "save_data": true
}

```

xt 本身就是跨多个文件组成一张图，因此使用 `time_range_fs + stride`，不使用 `select`。

8. particle_phase: 粒子相空间

例如电子的 `x-px`:

```

{
  "type": "particle_phase",
  "name": "foil_x_px",
  "prefix": "foilparticles",
  "x_variable": {
    "contains": ["Grid_Particles"],
    "component": "x"
  },
  "y_variable": {
    "contains": ["Particles_Px"]
  },
  "weight_variable": {
    "contains": ["Particles_Weight"]
  },
  "x_normalization": {
    "factor": 1000000000.0,
    "label": "x (nm)"
  },
  "y_normalization": {
    "kind": "momentum_mec",
    "label": "$p_x/(m_e c)$"
  },
  "select": {
    "mode": "number",
    "number": [60, 65, 70, 75, 78]
  },
  "animate": false,
  "xlim": [360, 720],
  "ylim": [-20, 40],
  "bins": [400, 400],
  "scale": "log",
  "save_data": true
}

```

与 `spatial` 一样, `particle_phase + animate=false` 支持单个或多个 `time_fs/number`。

9. particle_xt: 粒子位置-时间分布

示例:

```
{
  "type": "particle_xt",
  "name": "foil_particle_xt",
  "prefix": "foilparticles",
  "position_variable": {
    "contains": ["Grid_Particles"],
    "component": "x"
  },
  "weight_variable": {
    "contains": ["Particles_Weight"]
  },
  "position_normalization": {
    "factor": 1000000000.0,
    "label": "x (nm)"
  },
  "time_range_fs": [0.0, null],
  "stride": 1,
  "space_range": [360, 720],
  "bins": 500,
  "scale": "log",
  "save_data": true
}
```

10. 变量选择方式

精确变量名

如果已经知道完整变量名:

```
"variable": {
  "name": " 完整的 _sdf_xarray_ 变量名"
}
```

关键词自动匹配

推荐通常使用:

```
"variable": {
  "contains": ["Number_Density", "Electron_foil"]
}
```

程序要求这些关键词最终只能唯一匹配一个变量。如果匹配到 0 个或多个, 程序会打印候选变量并报错。

11. INSPECT: 不知道变量名时怎么办

把:

```
"inspect": {
  "enabled": true,
  "prefix": "foilparticles",
  "particle_file": true,
  "select": {
    "mode": "number",
    "number": 78
  }
}
```

设置为 `true` 后运行程序。

程序会打印这个文件中的:

- 变量名称;
- dimensions;
- shape。

确认变量名之后, 再把 `inspect.enabled` 改回 `false`。

对于粒子 SDF, 必须:

```
"particle_file": true
```

这样 `sdf_xarray` 会使用 `keep_particles=True`。

12. 归一化

程序内置:

密度

```
"normalization": {
  "kind": "density_nc"
}
```

对应:

n / n_c

其中 `nc` 根据 `laser_wavelength_nm` 自动计算。

电流

```
"normalization": {
  "kind": "current_j0"
}
```

对应:

$J / (e \hbar c)$

动量

```
"normalization": {  
  "kind": "momentum_mec"  
}
```

对应:

$p / (m_e c)$

自定义乘数

例如 $m \rightarrow nm$:

```
"x_normalization": {  
  "factor": 1000000000.0,  
  "label": "x (nm)"  
}
```

不归一化

可以省略 `normalization`, 或者写:

```
"normalization": {  
  "kind": "none"  
}
```

13. 色标

线性色标

```
"scale": "linear"
```

对数色标

适合密度、粒子计数等正值:

```
"scale": "log"
```

可指定:

```
"vmin": 0.01,  
"vmax": 200
```

对称 symlog

适合正负电流、电场：

```
"scale": "symlog",  
"linthresh": 0.02
```

对称线性色标

```
"scale": "linear",  
"symmetric": true
```

自动范围

如果不写 vmin/vmax，程序使用 percentile 自动估计。默认：

```
"percentile": 99.5
```

14. 输出文件

每个 task 独立输出到：

```
<out_dir>/<task.name>/
```

例如：

```
Data/postprocess/foil_density_xy/
```

可能包含：

- .png: 静态图；
- .gif: 动画；
- .npz: save_data=true 时保存的后处理数据。

多时刻/多编号任务会额外输出一张 multiple_time 或 multiple_number 对比图，并使用同一个 colorbar 范围，便于直接比较。

15. 推荐的日常使用方式

以后不建议再复制很多不同版本的 Python 后处理脚本。建议：

统一程序：

```
~/python_lib/epochpost.py
```

每个算例：

```
run1/epochpost_input.json  
run2/epochpost_input.json  
run5/epochpost_input.json
```

run7/epochpost_input.json

...

每次后处理时只修改当前 run 的:

epochpost_input.json

如果一个 run 有很多不同分析, 可以让 tasks 中同时包含多个任务:

```
"tasks": [
  {"type": "spatial", "name": "density", "...": "..."},
  {"type": "spatial", "name": "Jx", "...": "..."},
  {"type": "xt", "name": "Jy_xt", "...": "..."},
  {"type": "particle_phase", "name": "x_px", "...": "..."}
]
```

这样一份 JSON 本身也相当于这个算例的“后处理记录”, 以后回头看 run7 时可以直接知道当时画过什么、用了什么范围和归一化。

16. 当前版本支持的任务类型总结

type	用途	单/多时刻	多编号	动图
spatial	2D 空间分布			
xt	空间-时间图	通过时间范围	不使用	—
particle_phase	粒子相空间			
particle_xt	粒子位置-时间图	通过时间范围	不使用	—

这套结构已经适合作为后续 Python library 的基础: 绘图算法集中在模块里, 算例差异全部留在 JSON 配置中。

GitHub 上传

建议 GitHub 仓库只保存程序、示例输入和说明文档, 不要提交大型 SDF 数据或后处理结果。

第一次上传一个新的本地仓库时, 可以在 GitHub 新建一个空仓库 epochpost, 然后在本地执行:

```
mkdir epochpost
```

```
cd epochpost
```

```
# 把这三个文件复制到当前目录:
# epochpost.py
# epochpost_input.json
# README.md
```

```
git init
git branch -M main
git add epochpost.py epochpost_input.json README.md
git commit -m "Initial release of epochpost"
git remote add origin https://github.com/YOUR_USERNAME/epochpost.git
git push -u origin main
```

之后每次更新只需要：

```
git add epochpost.py epochpost_input.json README.md
git commit -m "Update epochpost"
git push
```

对于实际 PIC 算例，建议把 Data/、*.sdf、postprocess/ 等大型数据加入 .gitignore，避免误上传模拟数据。

下一次更新功能：

1. FFT功能，能画FFT的结果